

vision globale

Gaelle KOALA

July 2026

Chapitre 1

Vision Globale du Projet et Stratégie de Développement

1.1 Objectif

L'objectif de ce document est de fournir une vision complète de l'organisation du projet Flight Control Computer afin de faciliter son développement.

Il présente :

- l'organisation complète du dépôt Git ;
- l'architecture logicielle du firmware ;
- le rôle de chaque dossier ;
- le rôle de chaque module logiciel ;
- l'ordre recommandé de développement.

Ce document servira de feuille de route pendant toute la réalisation du projet.

1.2 Organisation générale du dépôt Git

Flight-Control-Computer/

docs/

firmware/

hardware/

simulation/

tests/

tools/

README.md

1.3 Description des dossiers principaux

Dossier	Description
docs	Contient toute la documentation du projet (architecture, conception, tests, manuel utilisateur...).
firmware	Contient l'ensemble du code embarqué exécuté sur le STM32.
hardware	Schémas électroniques, brochage, alimentation, PCB et composants.
simulation	Modèles MATLAB, Simulink, XFLR5 et simulations de contrôle.
tests	Tests unitaires, tests d'intégration et procédures de validation.
tools	Scripts Python, outils de génération, convertisseurs et utilitaires.
README.md	Présentation générale du projet et procédure d'installation.

1.4 Architecture du Firmware

Le firmware est organisé en plusieurs couches logicielles.

```
firmware/  
  
Core/  
BSP/  
Drivers/  
Interfaces/  
Config/  
Services/  
Estimation/  
Control/  
Application/  
RTOS/  
Middleware/  
Tests/
```

1.5 Description des dossiers du firmware

1.5.1 Core

Cette partie est générée automatiquement par STM32CubeMX.
Elle contient notamment :

- main.c
- stm32xx_i.t.c
- system_{stm32xx}.c

— startup.s

Son rôle est de démarrer le microcontrôleur.

1.5.2 BSP (Board Support Package)

Le BSP regroupe tout ce qui dépend de la carte STM32 utilisée.

Exemples :

— configuration GPIO ;

— horloge système ;

— timers ;

— configuration des broches.

Si une nouvelle carte STM32 est utilisée, seul ce dossier doit être modifié.

1.5.3 Drivers

Les drivers permettent de communiquer avec les composants matériels.

Organisation proposée :

Drivers/

MPU6050/

BMP280/

FlySky/

PWM/

Battery/

UART/

SPI/

I2C/

DMA/

Chaque driver possède un fichier .c et un fichier .h.

Responsabilité :

— initialiser le périphérique ;

— lire les données ;

— écrire les données ;

— ne contenir aucune logique métier.

1.5.4 Interfaces

Ce dossier contient uniquement les structures de données communes.

Exemples :

imu_data.h

bmp_data.h

aircraft_state.h

flight_command.h

servo_command.h

battery_data.h

health_status.h

Aucun algorithme n'est présent dans ce dossier.

1.5.5 Config

Contient tous les paramètres du système.

Exemples :

- gains PID ;
- fréquences ;
- constantes physiques ;
- paramètres avion.

1.5.6 Services

Les services regroupent les fonctions génériques réutilisables.

Organisation proposée :

Services/

 Logger/

 Parameters/

 Time/

 Math/

 Filters/

 Diagnostics/

 Health/

 Utilities/

1.5.7 Estimation

Cette couche estime l'état de l'aéronef.

Organisation :

Estimation/

AHRS/

Complementary/

EKF/

StateEstimator/

Barometer/

Responsabilités :

- calcul du Roll ;
- calcul du Pitch ;
- calcul du Yaw ;
- estimation de l'altitude ;
- estimation de la vitesse verticale.

1.5.8 Control

Implémente les lois de commande.

Organisation :

Control/

PID/

Roll/

Pitch/

Yaw/

Altitude/

Mixer/

Responsabilités :

- calcul des commandes ;
- régulation ;
- mélange des commandes.

1.5.9 Application

Cette couche représente l'intelligence du Flight Control Computer.

Organisation :

Application/

FlightManager/

FlightModes/

Navigation/

Mission/

StateMachine/

Failsafe/

Elle décide du comportement global de l'avion.

1.5.10 RTOS

Cette couche contient tous les éléments liés à FreeRTOS.

RTOS/

Tasks/

Queues/

Mutexes/

Events/

Notifications/

Timers/

Scheduler/

Hooks/

1.5.11 Middleware

Bibliothèques externes.

Exemples :

- FreeRTOS
- CMSIS
- futures bibliothèques GPS

1.5.12 Tests

Tests unitaires et tests d'intégration de chaque module logiciel.

1.6 Dépendances entre les couches

Le logiciel respecte les dépendances suivantes :

Application

↓

Control

↓

Estimation

↓

Drivers

↓

STM32 HAL

↓

Microcontrôleur

Une couche inférieure ne doit jamais dépendre d'une couche supérieure.

1.7 Ordre recommandé de développement

Le développement sera réalisé progressivement.

1. Création du projet STM32CubeMX.
2. Initialisation du dépôt Git.
3. Mise en place de l'arborescence du projet.
4. Création des structures de données (Interfaces).
5. Configuration du BSP.

6. Développement des Drivers.
7. Validation individuelle de chaque Driver.
8. Mise en place de FreeRTOS.
9. Création des tâches principales.
10. Développement de la couche Estimation.
11. Développement de la couche Control.
12. Développement du Mixer.
13. Développement de l'Application.
14. Développement des modes de vol.
15. Développement du Failsafe.
16. Développement de la télémétrie.
17. Développement de la journalisation.
18. Tests d'intégration.
19. Validation sur banc.
20. Validation sur avion.

1.8 Ordre de codage des Drivers

Les drivers seront développés dans l'ordre suivant :

1. GPIO
2. UART
3. PWM
4. ADC
5. I2C
6. SPI
7. BMP280
8. MPU6050
9. FlySky iBus
10. Batterie

Chaque driver devra être testé indépendamment avant d'être intégré au reste du système.

1.9 Conclusion

Cette architecture permet de séparer clairement les responsabilités des différents modules logiciels.

Le développement pourra être réalisé de manière progressive en validant chaque couche avant de passer à la suivante. Cette approche garantit un code modulaire, facilement maintenable et évolutif, tout en limitant les risques d'intégration.